

1	Технология CUDA: алгоритм вычисления гистограммы (тип данных – float). Преимущества и недостатки представленной реализации	3
2	Технология CUDA: алгоритм операции инклюзивного scan (ограничения: размер матрицы – $M \times N$, тип данных – int, операция применяется к каждой строке независимо). Преимущества и недостатки представленной реализации	4
3	Технология CUDA: алгоритм операции эксклюзивного scan (ограничения: размер матрицы – $M \times N$, тип данных – int, операция применяется к каждой строке независимо). Преимущества и недостатки представленной реализации	5
4	Технология CUDA: алгоритм преобразования цветного изображения в оттенки серого (структура в памяти – RGBRGBRGBRGB..., тип данных – ushort, формула преобразования – среднее арифметическое). Преимущества и недостатки представленной реализации	6
5	Технология CUDA: алгоритм преобразования цветного изображения в оттенки серого (структура в памяти – RGBRGBRGBRGB..., тип данных – uchar, формула преобразования – среднее арифметическое). Преимущества и недостатки представленной реализации	7
6	Технология CUDA: алгоритм редукции (ограничения: размер матрицы – $M \times N$, тип данных – int, операция применяется к каждой строке независимо). Преимущества и недостатки представленной реализации	8
7	Технология CUDA: алгоритм свертки. Преимущества и недостатки представленной реализации.....	9
8	Технология CUDA: асинхронное и синхронное копирование в CUDA на примере суммирования двух векторов. Pinned память. Способы выделения	10
9	Технология CUDA: задание ядра, потоки и блоки потоков ядра для операции инклюзивного scan	12
10	Технология CUDA: задание ядра, формирование потоков и блока потоков ядра, структура вызова ядра на примере эксклюзивного scan	13
11	Технология CUDA: задание ядра, формирование потоков и блока потоков ядра, структура вызова ядра на примере операции редукции	14
12	Технология CUDA: задание ядра, формирование потоков и блока потоков ядра, структура вызова ядра на примере преобразования цветного изображения в оттенки серого	15
13	Технология CUDA: механизм транзакций в CUDA, примеры корректного и некорректного обращения в память на примере чтения цветного изображения. Вычислить число транзакций для представленных примеров	16
14	Технология CUDA: понятие оссурансу, зависимость от аппаратных блоков, пример расчета	17
15	Технология CUDA: пример организации конфликта по банкам в разделяемой памяти на примере чтения цветного изображения	18

16	Технология CUDA: пример правильной инициализации матрицы $M \times N$ для последующего применения в ядре на GPU. Преимущества и недостатки представленной реализации.....	19
17	Технология CUDA: синхронизация потоков, дивергенция потоков, операция перестановки между потоками (shuffles) на примере ядра эксклюзивного scan	20
18	Технология CUDA: синхронизация потоков, дивергенция потоков, операция перестановки между потоками (shuffles) на примере ядра инклюзивного scan	21
19	Технология CUDA: синхронизация потоков, дивергенция потоков, операция перестановки между потоками (shuffles) на примере ядра редукции.....	22
20	Технология CUDA: создание, инициализация и синхронизация CUDA Stream на примере суммирования двух матриц.....	23
21	Технология CUDA: структура ядра и адресация на примере ядра инклюзивного scan (тип данных, передаваемый в ядро, – int, фактический тип данных – char)	24
22	Технология CUDA: структура ядра и адресация на примере ядра преобразования цветного изображения в оттенки серого (тип данных, передаваемый в ядро, – int, фактический тип данных – char).....	25
23	Технология CUDA: структура ядра и адресация на примере ядра эксклюзивного scan (тип данных, передаваемый в ядро, – int, фактический тип данных – char)	26
24	Технология CUDA: типы памяти в CUDA, примеры создания и организации доступа на примере операции свертки	27
25	Технология CUDA: формирование потоков и блока потоков, вычисление смещения в ядре (тип данных – char, 1 поток обрабатывает 2×6 элементов типа int)	28
26	Технология CUDA: формирование потоков и блока потоков, вычисление смещения в ядре (тип данных – short, 1 поток обрабатывает 2×6 элементов типа short)	29
27	Технология CUDA: формирование потоков и блока потоков, вычисление смещения в ядре (тип данных – short, 1 поток обрабатывает 3×1 элементов типа short)	30
28	Технология CUDA: формирование потоков и блока потоков, вычисление смещения в ядре (тип данных – char, 1 поток обрабатывает 4×6 элементов типа int)	31
29	Технология CUDA: формирование потоков и блока потоков, вычисление смещения в ядре (тип данных – short, 1 поток обрабатывает 2×6 элементов типа short)	32
30	Технология CUDA: формирование потоков и блока потоков, вычисление смещения в ядре (тип данных – int, 1 поток обрабатывает 1×1 элемент типа int).....	33
31	Технология CUDA: формирование потоков и блока потоков, вычисление смещения в ядре (тип данных – short, 1 поток обрабатывает 3×3 элементов типа int)	34

1 Технология CUDA: алгоритм вычисления гистограммы (тип данных – float). Преимущества и недостатки представленной реализации

cuda

```
__global__ void histogram_float(const float* data, int* hist,
                                int n, float minv, float maxv, int bins) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;
    float x = data[i];
    int b = (int)((x - minv) / (maxv - minv)) * bins;
    if (b < 0) b = 0;
    if (b >= bins) b = bins - 1;
    atomicAdd(&hist[b], 1);
}
```

// Преимущества: простой код, легко адаптировать под другой тип/кол-во корзинок.

// Недостатки: сильный contention по atomicAdd при неравномерном распределении.

2 Технология CUDA: алгоритм операции инклюзивного scan (ограничения: размер матрицы – $M \times N$, тип данных – int, операция применяется к каждой строке независимо). Преимущества и недостатки представленной реализации

cuda

```
__global__ void inclusive_scan_rows(const int* in, int* out, int M, int N)
{
    extern __shared__ int s[];
    int row = blockIdx.x;
    int tid = threadIdx.x;
    int col = tid;
    if (row >= M || col >= N) return;

    s[tid] = in[row * N + col];
    __syncthreads();

    for (int off = 1; off < blockDim.x; off <= 1) {
        int val = 0;
        if (tid >= off) val = s[tid - off];
        __syncthreads();
        if (tid >= off) s[tid] += val;
        __syncthreads();
    }

    out[row * N + col] = s[tid];
}
```

// Преимущества: классический Blelloch-like scan в shared memory, логарифмическая сложность.

// Недостатки: одна строка = один блок, ограничение по N и shared memory.

3 Технология CUDA: алгоритм операции эксклюзивного scan (ограничения: размер матрицы – $M \times N$, тип данных – int, операция применяется к каждой строке независимо). Преимущества и недостатки представленной реализации

cuda

```
__global__ void exclusive_scan_rows(const int* in, int* out, int M, int N)
{
    extern __shared__ int s[];
    int row = blockIdx.x;
    int tid = threadIdx.x;
    int col = tid;
    if (row >= M || col >= N) return;

    s[tid] = in[row * N + col];
    __syncthreads();

    int prev = s[tid];
    s[tid] = 0;
    __syncthreads();

    for (int off = 1; off < blockDim.x; off <= 1) {
        int val = 0;
        if (tid >= off) val = s[tid - off];
        __syncthreads();
        if (tid >= off) s[tid] += val;
        __syncthreads();
    }

    out[row * N + col] = s[tid];
}
```

// Преимущества: прямое получение эксклюзивного результата, тот же шаблон, что и для инклюзивного.

// Недостатки: те же ограничения по длине строки и объёму shared memory.

4 Технология CUDA: алгоритм преобразования цветного изображения в оттенки серого (структура в памяти – RGBRGBRGB..., тип данных – ushort, формула преобразования – среднее арифметическое). Преимущества и недостатки представленной реализации

cuda

```
__global__ void rgb_to_gray_ushort(const ushort* rgb,
                                   ushort* gray, int pixels) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= pixels) return;
    int base = 3 * i;
    ushort r = rgb[base + 0];
    ushort g = rgb[base + 1];
    ushort b = rgb[base + 2];
    gray[i] = (ushort)((r + g + b) / 3);
}
```

// Преимущества: минимальная арифметика, полностью целочисленный путь.

// Недостатки: потенциальное переполнение при сумме (требуется более широкий тип).

5 Технология CUDA: алгоритм преобразования цветного изображения в оттенки серого (структура в памяти – RGBRGBRGB..., тип данных – uchar, формула преобразования – среднее арифметическое). Преимущества и недостатки представленной реализации

cuda

```
__global__ void rgb_to_gray_uchar(const uchar* rgb,
                                   uchar* gray, int pixels) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= pixels) return;
    int base = 3 * i;
    uchar r = rgb[base + 0];
    uchar g = rgb[base + 1];
    uchar b = rgb[base + 2];
    gray[i] = (uchar)((r + g + b) / 3);
}
```

// Преимущества: дешёвая целочисленная арифметика, компактные данные.

// Недостатки: нет учёта реальной яркости (нет весов), простое среднее даёт низкое качество.

6 Технология CUDA: алгоритм редукции (ограничения: размер матрицы – $M \times N$, тип данных – `int`, операция применяется к каждой строке независимо). Преимущества и недостатки представленной реализации

cuda

```
__global__ void reduce_rows(const int* in, int* out, int M, int N) {
    extern __shared__ int s[];
    int row = blockIdx.x;
    int tid = threadIdx.x;
    int col = tid;

    if (row >= M) return;
    s[tid] = (col < N) ? in[row * N + col] : 0;
    __syncthreads();

    for (int off = blockDim.x / 2; off > 0; off >>= 1) {
        if (tid < off) s[tid] += s[tid + off];
        __syncthreads();
    }

    if (tid == 0) out[row] = s[0];
}
```

// Преимущества: логарифмический по N алгоритм, используемый shared memory.

// Недостатки: ограничение по длине строки, возможные bank-conflicts в shared memory.

7 Технология CUDA: алгоритм свертки. Преимущества и недостатки представленной реализации

cuda

```
__global__ void conv2d(const float* img, const float* kernel,
                      float* out, int W, int H, int K) {
    extern __shared__ float tile[];
    int tx = threadIdx.x, ty = threadIdx.y;
    int x = blockIdx.x * blockDim.x + tx;
    int y = blockIdx.y * blockDim.y + ty;

    // Загрузка тайла без сложных «халов»
    if (x < W && y < H)
        tile[ty * blockDim.x + tx] = img[y * W + x];
    else
        tile[ty * blockDim.x + tx] = 0;
    __syncthreads();

    if (x >= W || y >= H) return;

    float acc = 0.0f;
    int r = K / 2;
    for (int ky = -r; ky <= r; ++ky)
        for (int kx = -r; kx <= r; ++kx) {
            int ix = tx + kx;
            int iy = ty + ky;
            if (ix >= 0 && iy >= 0 &&
                ix < blockDim.x && iy < blockDim.y) {
                float v = tile[iy * blockDim.x + ix];
                float kv = kernel[(ky + r) * K + (kx + r)];
                acc += v * kv;
            }
        }
    out[y * W + x] = acc;
}
```

// Преимущества: переиспользование данных через shared memory, уменьшение обращений к глобальной памяти.

// Недостатки: неполная/грубая обработка краёв, сложная индексация для больших K.

8 Технология CUDA: асинхронное и синхронное копирование в CUDA на примере суммирования двух векторов. Pinned память. Способы выделения cuda

```
__global__ void vec_add(const float* a, const float* b,
                        float* c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}
```

// Синхронное копирование

```
void run_sync(float* h_a, float* h_b, float* h_c, int n) {
    float *d_a, *d_b, *d_c;
    size_t sz = n * sizeof(float);
    cudaMalloc(&d_a, sz);
    cudaMalloc(&d_b, sz);
    cudaMalloc(&d_c, sz);

    cudaMemcpy(d_a, h_a, sz, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, h_b, sz, cudaMemcpyHostToDevice);
    vec_add<<<(n+255)/256, 256>>>(d_a, d_b, d_c, n);
    cudaMemcpy(h_c, d_c, sz, cudaMemcpyDeviceToHost);

    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
}
```

// Асинхронное копирование + pinned

```
void run_async(int n) {
    float *h_a, *h_b, *h_c;
    size_t sz = n * sizeof(float);
    cudaMallocHost(&h_a, sz); // pinned
    cudaMallocHost(&h_b, sz);
    cudaMallocHost(&h_c, sz);

    float *d_a, *d_b, *d_c;
    cudaMalloc(&d_a, sz);
    cudaMalloc(&d_b, sz);
    cudaMalloc(&d_c, sz);

    cudaStream_t s;
    cudaStreamCreate(&s);
}
```

```
cudaMemcpyAsync(d_a, h_a, sz, cudaMemcpyHostToDevice, s);
cudaMemcpyAsync(d_b, h_b, sz, cudaMemcpyHostToDevice, s);
vec_add<<<(n+255)/256, 256, 0, s>>>(d_a, d_b, d_c, n);
cudaMemcpyAsync(h_c, d_c, sz, cudaMemcpyDeviceToHost, s);
cudaStreamSynchronize(s);

cudaStreamDestroy(s);
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
cudaFreeHost(h_a); cudaFreeHost(h_b); cudaFreeHost(h_c);
}
```

// Преимущества pinned: высокая скорость H2D/D2H и возможность Async.

// Недостатки: pinned память «дорога» и ограничена, фрагментирует ОЗУ.

9 Технология CUDA: задание ядра, потоки и блоки потоков ядра для операции инклюзивного scan

cuda

// Ядро (пример): inclusive_scan_rows из выше

int M, N;

dim3 block(N); *// предположим $N \leq$ макс. потоков*

dim3 grid(M);

size_t sh = N * **sizeof(int)**;

inclusive_scan_rows<<<grid, block, sh>>>(d_in, d_out, M, N);

// Преимущества: простое соответствие «1 строка = 1 блок».

// Недостатки: не масштабируется, если N большое (>1024).

10 Технология CUDA: задание ядра, формирование потоков и блока потоков ядра, структура вызова ядра на примере эксклюзивного scan
cuda

// Ядро: exclusive_scan_rows (выше)

int M, N;

int threads = 256;

dim3 block(threads);

dim3 grid(M); *// 1 блок на строку*

size_t sh = threads * **sizeof(int)**;

exclusive_scan_rows<<<grid, block, sh>>>(d_in, d_out, M, N);

// Преимущества: стандартный шаблон Blelloch, удобно для фиксированного N.

// Недостатки: для $N > threads$ требуется несколько проходов/разбиение строки.

11 Технология CUDA: задание ядра, формирование потоков и блока потоков ядра, структура вызова ядра на примере операции редукции
cuda

// Ядро: reduce_rows (выше)

int M, N;

int threads = 256;

dim3 block(threads);

dim3 grid(M);

size_t sh = threads * **sizeof(int)**;

reduce_rows<<<grid, block, sh>>>(d_in, d_out, M, N);

// Преимущества: один блок полностью обрабатывает строку.

// Недостатки: ограничение по N и количеству регистров/SM.

12 Технология CUDA: задание ядра, формирование потоков и блока потоков ядра, структура вызова ядра на примере преобразования цветного изображения в оттенки серого

cuda

// Ядро: rgb_to_gray_uchar или rgb_to_gray_ushort

int pixels;

int threads = 256;

dim3 block(threads);

dim3 grid((pixels + threads - 1) / threads);

rgb_to_gray_uchar<<<grid, block>>>(d_rgb, d_gray, pixels);

// Преимущества: 1 поток = 1 пиксель, хорошая параллельность.

// Недостатки: возможная неидеальная когерентность при 3-байтовом RGB.

13 Технология CUDA: механизм транзакций в CUDA, примеры корректного и некорректного обращения в память на примере чтения цветного изображения. Вычислить число транзакций для представленных примеров

cuda

// Некорректный доступ (scatter): каждый поток читает несоседние байты

```
__global__ void bad_rgb_read(const uchar* rgb, uchar* out, int pixels) {  
    int i = blockIdx.x * blockDim.x + threadIdx.x;  
    if (i >= pixels) return;  
    out[i] = rgb[i * 3]; // R компонента: шаг 3 байта  
}
```

// Корректный доступ (coalesced): потоки читают подряд

```
__global__ void good_rgb_read(const uchar* rgb, uchar* r, uchar* g, uchar*  
b, int pixels) {  
    int i = blockIdx.x * blockDim.x + threadIdx.x;  
    if (i >= pixels) return;  
    int base = 3 * i;  
    r[i] = rgb[base + 0];  
    g[i] = rgb[base + 1];  
    b[i] = rgb[base + 2];  
}
```

// При размере warp 32 и выровненных адресах:

// good_rgb_read: 1–2 транзакции (зависит от выравнивания).

// bad_rgb_read: до 3–4 транзакций на warp (нерациональная выборка).

14 Технология CUDA: понятие occupancy, зависимость от аппаратных блоков, пример расчета

cuda

// Условный пример: часть расчёта occupancy вручную

int maxThreadsPerSM = 2048;

int warpSize = 32;

int regsPerSM = 65536;

int regsPerThread = 64;

int shmemPerSM = 48 * 1024;

int shmemPerBlock = 8 * 1024;

int threadsPerBlock = 256;

int maxByThreads = maxThreadsPerSM / threadsPerBlock; *// 8 блоков*

int maxByRegs = regsPerSM / (regsPerThread * threadsPerBlock); *// 4*

блока

int maxByShmem = shmemPerSM / shmemPerBlock; *// 6 блоков*

int activeBlocks = min(min(maxByThreads, maxByRegs), maxByShmem);

// 4

float occupancy = (**float**)(activeBlocks * threadsPerBlock) /
(**float**)maxThreadsPerSM; *// 4*256/2048 = 0.5 (50%)*

// Зависимость: лимитируется регистрами, shared memory и потоками на SM.

15 Технология CUDA: пример организации конфликта по банкам в разделяемой памяти на примере чтения цветного изображения

cuda

```
__global__ void bank_conflict_example(const uchar* rgb, uchar* out) {  
    __shared__ uchar tile[32 * 32]; // 32 банка
```

```
    int tid = threadIdx.x;          // 0..31
```

*// Каждый поток читает в одну и ту же позицию по модулю размера
банка*

```
    tile[tid * 32] = rgb[tid];      // все пишут в один и тот же банк
```

```
    __syncthreads();
```

```
    out[tid] = tile[tid * 32];
```

```
}
```

// Преимущество: демонстрация конфликта.

// Недостаток: максимальный конфликт по банкам, 32x замедление.

16 Технология CUDA: пример правильной инициализации матрицы $M \times N$ для последующего применения в ядре на GPU. Преимущества и недостатки представленной реализации

cuda

```
__global__ void init_matrix(int* A, int M, int N, int pitchN) {  
    int x = blockIdx.x * blockDim.x + threadIdx.x; // столбец  
    int y = blockIdx.y * blockDim.y + threadIdx.y; // строка  
    if (x >= N || y >= M) return;  
    A[y * pitchN + x] = y * N + x;  
}
```

// Преимущества: инициализация сразу на GPU, coalesced по x.

// Недостатки: необходимость следить за pitch/step, возможное различие N и pitchN.

17 Технология CUDA: синхронизация потоков, дивергенция потоков, операция перестановки между потоками (shuffles) на примере ядра эксклюзивного scan

cuda

```
__global__ void exclusive_scan_warp(int* data, int n) {  
    int tid = threadIdx.x;  
    int lane = tid & 31;  
    int warpId = tid >> 5;  
    int x = data[tid];
```

// инклюзивный scan внутри warp

```
    for (int off = 1; off < 32; off <= 1) {  
        int y = __shfl_up_sync(0xffffffff, x, off);  
        if (lane >= off) x += y; // дивергенция по if, но внутри warp ок  
    }
```

// сделать эксклюзивный

```
    int excl = x - data[tid];  
    data[tid] = excl;  
}
```

// Преимущества: warp-операции не требуют __syncthreads, очень быстры.

// Недостатки: работает только внутри warp, нужна доп. логика для всего блока.

18 Технология CUDA: синхронизация потоков, дивергенция потоков, операция перестановки между потоками (shuffles) на примере ядра инклюзивного scan

cuda

```
__global__ void inclusive_scan_warp(int* data, int n) {
```

```
    int tid = threadIdx.x;
```

```
    int lane = tid & 31;
```

```
    int x = data[tid];
```

```
    for (int off = 1; off < 32; off <= 1) {
```

```
        int y = __shfl_up_sync(0xffffffff, x, off);
```

```
        if (lane >= off) x += y;    // ветвление внутри warp (легкая  
дивергенция)
```

```
    }
```

```
    data[tid] = x;
```

```
}
```

// Преимущества: минимальный overhead, нет глобального shared.

// Недостатки: не масштабируется автоматически на >32 элементов.

19 Технология CUDA: синхронизация потоков, дивергенция потоков, операция перестановки между потоками (shuffles) на примере ядра редукции

cuda

```
__global__ void reduce_warp(float* data) {  
    int tid = threadIdx.x;  
    int lane = tid & 31;  
    float x = data[tid];  
  
    // редукция в пределах warp  
    for (int off = 16; off > 0; off >>= 1) {  
        float y = __shfl_down_sync(0xffffffff, x, off);  
        x += y; // часть потоков простаивает -> дивергенция по сути  
    }  
  
    if (lane == 0) data[tid >> 5] = x; // по одному результату на warp  
}
```

// Преимущества: очень быстрая редукция внутри warp без shared.

// Недостатки: нужен дополнительный шаг редукции результатов warp'ов.

20 Технология CUDA: создание, инициализация и синхронизация CUDA Stream на примере суммирования двух матриц

cuda

```
__global__ void mat_add(const float* A, const float* B,
                        float* C, int M, int N) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= N || y >= M) return;
    int idx = y * N + x;
    C[idx] = A[idx] + B[idx];
}
```

```
void mat_add_streams(float* h_A, float* h_B, float* h_C, int M, int N) {
    size_t sz = M * N * sizeof(float);
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, sz);
    cudaMalloc(&d_B, sz);
    cudaMalloc(&d_C, sz);

    cudaStream_t s;
    cudaStreamCreate(&s);

    cudaMemcpyAsync(d_A, h_A, sz, cudaMemcpyHostToDevice, s);
    cudaMemcpyAsync(d_B, h_B, sz, cudaMemcpyHostToDevice, s);

    dim3 block(16, 16);
    dim3 grid((N + 15)/16, (M + 15)/16);
    mat_add<<<grid, block, 0, s>>>(d_A, d_B, d_C, M, N);

    cudaMemcpyAsync(h_C, d_C, sz, cudaMemcpyDeviceToHost, s);
    cudaStreamSynchronize(s);

    cudaStreamDestroy(s);
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
}
```

21 Технология CUDA: структура ядра и адресация на примере ядра инклюзивного scan (тип данных, передаваемый в ядро, – int, фактический тип данных – char)

cuda

```
__global__ void inclusive_scan_char(int* buf, int n) {  
    char* data = (char*)buf; // фактические данные = char  
    int tid = threadIdx.x;  
    if (tid >= n) return;  
  
    int x = (int)data[tid];  
  
    for (int off = 1; off < blockDim.x; off <= 1) {  
        int y = __shfl_up_sync(0xffffffff, x, off);  
        if (tid >= off) x += y;  
    }  
    data[tid] = (char)x;  
}
```

// Преимущества: можно упаковывать char в int буферы (для выравнивания).

// Недостатки: легко ошибиться с индексами / выравниванием.

22 Технология CUDA: структура ядра и адресация на примере ядра преобразования цветного изображения в оттенки серого (тип данных, передаваемый в ядро, – int, фактический тип данных – char)

cuda

```
__global__ void gray_from_int_buffer(int* rgbBuf, int* grayBuf, int
pixels) {
    char* rgb = (char*)rgbBuf;
    char* gray = (char*)grayBuf;

    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= pixels) return;

    int base = 3 * i;
    uchar r = (uchar)rgb[base + 0];
    uchar g = (uchar)rgb[base + 1];
    uchar b = (uchar)rgb[base + 2];
    gray[i] = (char)((r + g + b) / 3);
}
```

23 Технология CUDA: структура ядра и адресация на примере ядра эксклюзивного scan (тип данных, передаваемый в ядро, – int, фактический тип данных – char)

cuda

```
__global__ void exclusive_scan_char_from_int(int* buf, int n) {  
    char* data = (char*)buf;  
    int tid = threadIdx.x;  
    if (tid >= n) return;  
  
    int x = (int)data[tid];  
    int orig = x;  
  
    for (int off = 1; off < blockDim.x; off <= 1) {  
        int y = __shfl_up_sync(0xffffffff, x, off);  
        if (tid >= off) x += y;  
    }  
  
    int inclusive = x;  
    int exclusive = inclusive - orig;  
    data[tid] = (char)exclusive;  
}
```

24 Технология CUDA: типы памяти в CUDA, примеры создания и организации доступа на примере операции свертки

cuda

// Глобальная память

__constant__ float d_kernel[25]; *// константная память (напр. 5x5)*

__global__ void conv2d_types(**const float*** __restrict__ img,
 float* out, **int** W, **int** H, **int** K) {

extern __shared__ float tile[]; *// shared memory*

int tx = threadIdx.x, ty = threadIdx.y;

int x = blockIdx.x * blockDim.x + tx;

int y = blockIdx.y * blockDim.y + ty;

if (x < W && y < H)

 tile[ty * blockDim.x + tx] = img[y * W + x]; *// глобальная → shared*

else

 tile[ty * blockDim.x + tx] = 0;

 __syncthreads();

if (x >= W || y >= H) **return**;

float acc = 0.0f;

int r = K / 2;

for (**int** ky = -r; ky <= r; ++ky)

for (**int** kx = -r; kx <= r; ++kx) {

int ix = tx + kx;

int iy = ty + ky;

if (ix >= 0 && iy >= 0 &&

 ix < blockDim.x && iy < blockDim.y) {

float v = tile[iy * blockDim.x + ix]; *// shared*

float kv = d_kernel[(ky + r) * K + (kx + r)]; *// constant*

 acc += v * kv;

 }

 }

 out[y * W + x] = acc;

}

// Примеры: глобальная (img/out), shared (tile), constant (d_kernel), регистры (acc).

25 Технология CUDA: формирование потоков и блока потоков, вычисление смещения в ядре (тип данных – char, 1 поток обрабатывает 2x6 элементов типа int)

cuda

*// 2x6 int = 12 int = 48 байт, data хранится как char**

__global__ void kernel_char_2x6_int(**char*** data, **int** pitchBytes) {

int tx = threadIdx.x;

int ty = threadIdx.y;

int blockX = blockIdx.x * blockDim.x + tx;

int blockY = blockIdx.y * blockDim.y + ty;

int elemPerThreadX = 6;

int elemPerThreadY = 2;

// смещение в int

int offInt = (blockY * elemPerThreadY) * pitchBytes / **sizeof(int)** +
(blockX * elemPerThreadX);

// смещение в байтах

int offByte = offInt * **sizeof(int)**;

char* ptr = data + offByte;

// ... обработка 2x6 int через reinterpret_cast<int>(ptr)*

}

26 Технология CUDA: формирование потоков и блока потоков, вычисление смещения в ядре (тип данных – short, 1 поток обрабатывает 2x6 элементов типа short)

cuda

// 2x6 short = 12 short = 24 байта

__global__ void kernel_short_2x6(**short*** data, **int** pitchShort) {

int tx = threadIdx.x;

int ty = threadIdx.y;

int elemPerThreadX = 6;

int elemPerThreadY = 2;

int x0 = (blockIdx.x * blockDim.x + tx) * elemPerThreadX;

int y0 = (blockIdx.y * blockDim.y + ty) * elemPerThreadY;

int idx0 = y0 * pitchShort + x0;

short* ptr = data + idx0;

// ... обработка 2x6 short

}

27 Технология CUDA: формирование потоков и блока потоков, вычисление смещения в ядре (тип данных – short, 1 поток обрабатывает 3x1 элементов типа short)

cuda

// 3x1 short = 3 short

__global__ void kernel_short_3x1(**short*** data, **int** pitchShort) {

int tx = threadIdx.x;

int ty = threadIdx.y;

int elemPerThreadX = 3;

int elemPerThreadY = 1;

int x0 = (blockIdx.x * blockDim.x + tx) * elemPerThreadX;

int y0 = (blockIdx.y * blockDim.y + ty) * elemPerThreadY;

int idx = y0 * pitchShort + x0;

short* ptr = data + idx;

// ... обработка 3 short подряд

}

28 Технология CUDA: формирование потоков и блока потоков, вычисление смещения в ядре (тип данных – char, 1 поток обрабатывает 4x6 элементов типа int)

cuda

// 4x6 int = 24 int = 96 байт

__global__ void kernel_char_4x6_int(**char*** data, **int** pitchBytes) {

int tx = threadIdx.x;

int ty = threadIdx.y;

int elemPerThreadX = 6;

int elemPerThreadY = 4;

int x0_int = (blockIdx.x * blockDim.x + tx) * elemPerThreadX;

int y0_int = (blockIdx.y * blockDim.y + ty) * elemPerThreadY;

int offInt = y0_int * pitchBytes / **sizeof(int)** + x0_int;

int offByte = offInt * **sizeof(int)**;

char* ptr = data + offByte;

// ... обработка 4x6 int

}

29 Технология CUDA: формирование потоков и блока потоков, вычисление смещения в ядре (тип данных – short, 1 поток обрабатывает 2x6 элементов типа short)

cuda

// (повтор варианта 2x6 short, но можно другой pitch)

__global__ **void** kernel_short_2x6_v2(**short*** data, **int** pitchShort) {

int tx = threadIdx.x;

int ty = threadIdx.y;

int elemPerThreadX = 6;

int elemPerThreadY = 2;

int x0 = (blockIdx.x * blockDim.x + tx) * elemPerThreadX;

int y0 = (blockIdx.y * blockDim.y + ty) * elemPerThreadY;

short* ptr = data + y0 * pitchShort + x0;

// ... обработка 2x6 short

}

30 Технология CUDA: формирование потоков и блока потоков, вычисление смещения в ядре (тип данных – int, 1 поток обрабатывает 1x1 элемент типа int)

cuda

```
__global__ void kernel_int_1x1(int* data, int pitchInt, int M, int N) {  
    int x = blockIdx.x * blockDim.x + threadIdx.x; // столбец  
    int y = blockIdx.y * blockDim.y + threadIdx.y; // строка  
    if (x >= N || y >= M) return;  
  
    int idx = y * pitchInt + x;  
    int val = data[idx];  
    // ... обработка одного int  
}
```

31 Технология CUDA: формирование потоков и блока потоков, вычисление смещения в ядре (тип данных – short, 1 поток обрабатывает 3x3 элементов типа int)

cuda

// Матрица из int, но доступ через short (упакованный буфер)*

__global__ void kernel_short_3x3_int(**short*** buf, **int** pitchInts) {

int tx = threadIdx.x;

int ty = threadIdx.y;

int elemPerThreadX = 3;

int elemPerThreadY = 3;

int x0_int = (blockIdx.x * blockDim.x + tx) * elemPerThreadX;

int y0_int = (blockIdx.y * blockDim.y + ty) * elemPerThreadY;

int offInt = y0_int * pitchInts + x0_int;

int offShort = offInt * (**sizeof(int)** / **sizeof(short)**); *// 2 short на int*

short* ptr = buf + offShort;

// ... обработка 3x3 int через reinterpret_cast<int>(ptr)*

}
